MEMORANDUM

TO: Dr. Tina Han

FROM: Hengeltraub, Isabella; Wu, Xintong; Jhanwar, Yuvraj; Hassan, Khaleel

CC: ME32301 Lab Section 08- Harbin, Zachary; Alshaikh, Leen

DATE: March-4-2025

RE: Strain Gauge Rosettes and Stresses in Beams: Lab 5

EXECUTIVE SUMMARY

The main purpose of this lab is to understand the stress and strain of an I-shaped beam in a four-point bending configuration, as can be seen in Figure 1. The point of interest (poi) in this lab lies on (1, 1.75) from the center of the beam. Shear force (τ) and bending moment diagrams were created for this beam; shear force being the nonaxial forces divided by the beam cross section, and moment being the turning effect of the force on the body [1] - with a moment of 4P for the poi.

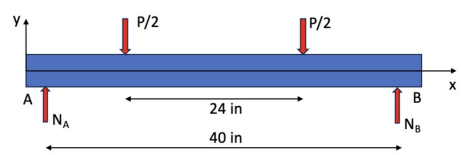

Figure 1: Beam Configuration

Next, σ_x (normal stress in x-direction), σ_y (normal stress in y-direction), and τ_{xy} (shear stress on x-face in y-direction) were calculated at poi for a load ranging from 0-8000, in intervals of 1000. σ_y and τ_{xy} were found to be zero since the point of interest lies on the surface of the beam, and has axial forces on x-direction only. σ_x was found using the equation 3 with $M(x)$ being the 4P bending moment and I_{zz} representing the moment of inertia of the I beam. I_{zz} was calculated using Equation 4, with variables in Table 1 (10.797 in^4).

$$\sigma_x(x, y) = \frac{-M(x)y}{I_{zz}} \tag{1}$$

$$I_{zz} = \frac{1}{12}t_w(h-2t_f)^3 + 2(\frac{1}{12}bt_f^3 + bt_f(\frac{h}{2} - \frac{t_f}{2})^2) \tag{2}$$

Depth (h) [in]	Flange Width (b) [in]	Web Thickness (t_w) [in]	Flange Thickness (t_f) [in]
4	4	.3675	.3675

Table 1: Moment of Inertia Values

Once those values were calculated, an online lab simulation was run at the poi for 3 strain gauge rosettes, which is a group of 3 strain gauges that give insight into how the beam is bending. Rosette 9-10-11 had angles [-45, 0, 45] relative to the z-axis, while rosette 12-13-14 had angles [-90, -45, 0] and rosette 15-16-16 had angles [-112.5, -67.5, -22.5] relative to the x-axis. Strain data was acquired for each gauge for loads 0-8000 at 1000 intervals. Then, each rosette's strain values and angles were run through a translation matrix, yielding values for ε_z, ε_x, and γ_{xz} for rosette 9-10-11, and values for ε_x, ε_y, and γ_{xy} for the other two rosettes. Each of these three result sets were then run along with the previously found σ_x, σ_y and τ_{xy} though another transformation matrix based on the generalized Hooke's law to find Poisson's ratio (ν), or the deformation of a material in one direction when loaded in another axis, and Young's modulus (E), or a measure of material stiffness. Since this matrix produced three equations for the two unknowns, the least squares method was employed. Finally, the confidence intervals for both variables were found with the bootstrap approach.

The ν and E for all three rosettes were within their 95% confidence ranges. With one exception, the percent error of all ν and Es were within 8% of expected values for E, and ν[2], as can be seen in tables 2-4. The exception was ν for rosette 15-16-17, with a percent error of about 38%, which although high, is still within its 95% confidence interval. Potential sources of error include rounding in MATLAB calculations, and slight errors in code, as the expiriment was conducted in an online database and the only errors could have come from data processing.

Property	**PE [%]**
E[GPa]	4.58
ν	3.7

Table 2: 9-10-11 Rosette

Property	**PE [%]**
E[GPa]	7.78
ν	5.55

Table 3: 12-13-14 Rosette

Property	**PE [%]**
E[GPa]	6.31
ν	38.27

Table 4: 15-16-17 Rosette

RESULTS

The strain was measured for the 9-10-11 rosette with the load increasing in increments of 1,000 lbs from 0 lb to 8,000 lbs, as shown in Table 5, where P represents the load, P_m represents the measured load, and ϵ_9, ϵ_{10}, and ϵ_{11} represents the measured strain for strain gauges 9, 10, and 11 respectively.

P [lb]	P_m [lb]	ϵ_9 $(*10^{-6})$	ϵ_{10} $(*10^{-6})$	ϵ_{11} $(*10^{-6})$
0	0	1	1	1
1000	1000	8	4	-28
2000	2000	16	9	-57
3000	3002	23	14	-88
4000	4002	29	19	-117
5000	4998	33	23	-147
6000	6000	36	26	-175
7000	7000	39	31	-204
8000	8000	40	35	-232

Table 5: Measured strain for strain gauges 9, 10, and 11.

Similarly, the strain was measured for the 12-13-14 rosette.

P [lb]	P_m [lb]	ϵ_{12} $(*10^{-6})$	ϵ_{13} $(*10^{-6})$	ϵ_{14} $(*10^{-6})$
0	0	1	1	1
1000	1000	-17	24	-28
2000	2000	-37	48	-56
3000	3001	-58	72	-81
4000	4001	-79	95	-105
5000	5002	-101	119	-126
6000	6001	-124	143	-146
7000	7002	-148	168	-164
8000	8000	-172	192	-179

Table 6: Measured strain for strain gauges 12, 13, and 14.

Again, the strain was measured for the 15-16-17 rosette.

P [lb]	P_m [lb]	ϵ_{15} $(*10^{-6})$	ϵ_{16} $(*10^{-6})$	ϵ_{17} $(*10^{-6})$
0	0	0	1	0
1000	1000	9	-10	-44
2000	2000	18	-22	-87
3000	3001	26	-34	-129
4000	4001	35	-46	-168
5000	5002	45	-58	-206
6000	6001	54	-70	-241
7000	7002	64	-83	-274
8000	8000	73	-96	-306

Table 7: Measured strain for strain gauges 15, 16, and 17.

The relationship between the gauge strains and the load applied are plotted in the graph below, which are all linear.

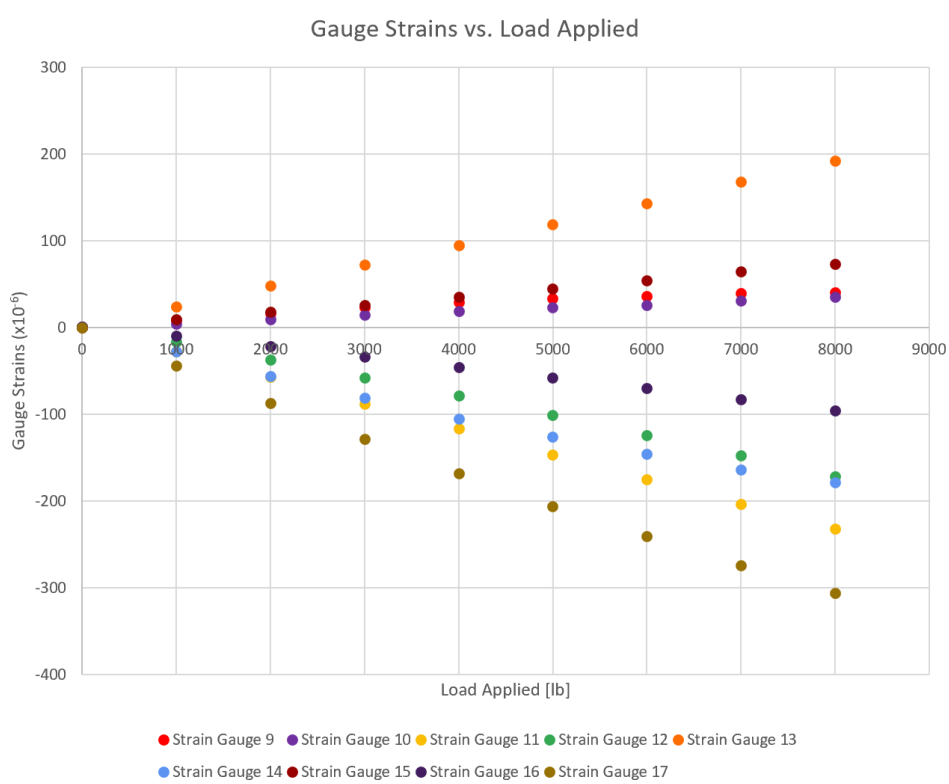

Figure 2: Graph depicting relationship between gauge strains and load

The shear force and bending moment diagram below illustrates the way the beam was loaded and the shear force and bending moment caused by the loads.

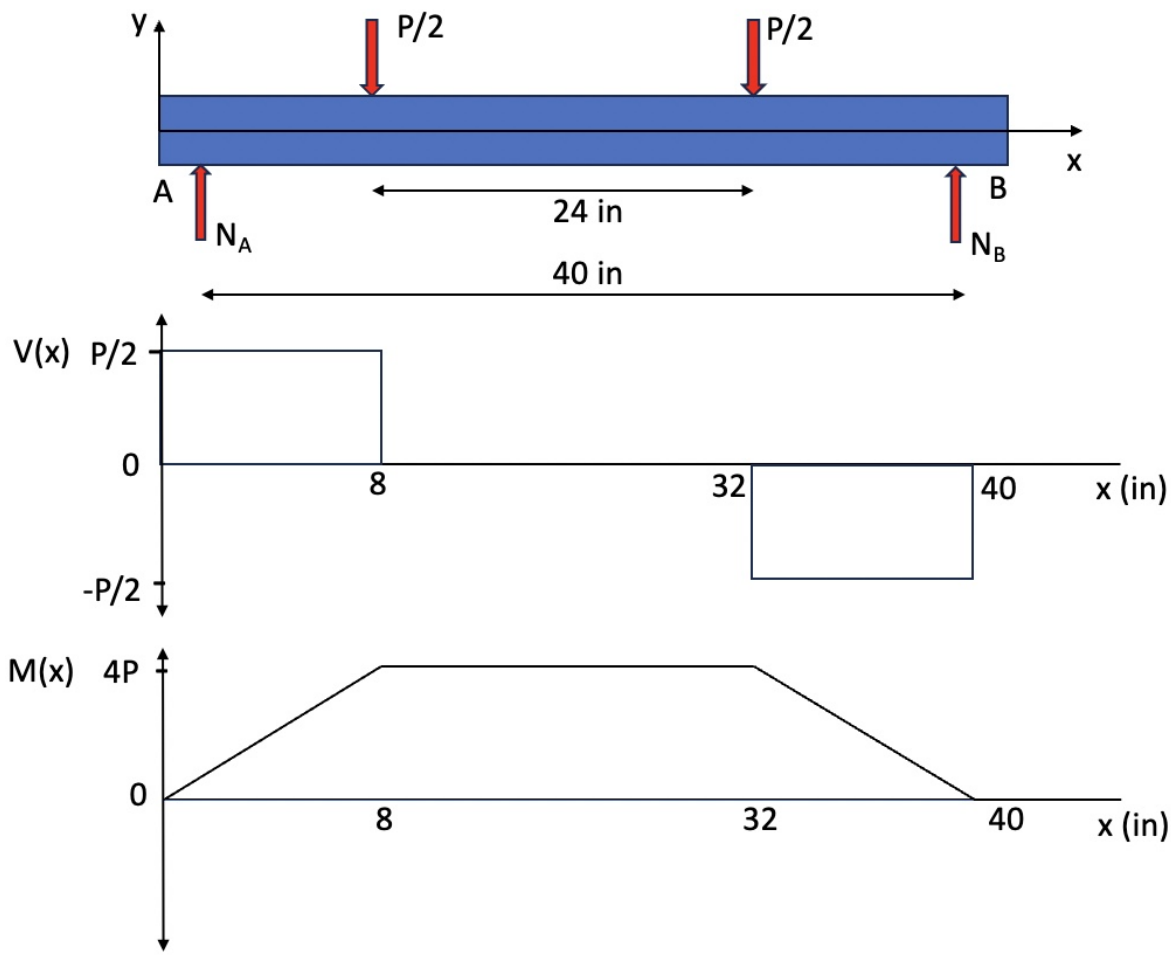

Figure 3: Shear force and bending moment diagram of beam

The shear force and bending moment diagram was used to calculate normal flexural stress using the following equation, with $M(x)$ representing the bending moment and I_{zz} representing the moment of inertia of the I-beam:

$$\sigma_x(x, y) = \frac{-M(x)y}{I_{zz}} \tag{3}$$

I_{zz} is calculated using the equation below, with t_w being the web thickness, h being the beam depth or height, t_f being the flange thickness, and b being the flange width.

$$I_{zz} = \frac{1}{12}t_w(h-2t_f)^3 + 2(\frac{1}{12}bt_f^3 + bt_f(\frac{h}{2} - \frac{t_f}{2})^2) \tag{4}$$

Additionally, the shear stress within the flange is calculated using the equation below, with $V(x)$ being shear force, Q_f being the first moment of area of the flange, and t being the flange thickness.

$$\tau_{xz}(x,s) = \frac{|V(x)|Q_f(s)}{I_{zz}t(s)} \tag{5}$$

$Q_f(s)$ is calculated using the following equation:

$$Q_f(s) = st_f(\frac{h}{2} - \frac{t_f}{2}) \tag{6}$$

Similarly, the shear stress within the web of the beam is calculated with Equation 7, with Q_f being the first moment of area of the web, and t being the web thickness.

$$\tau_{xy}(x,y) = \frac{|V(x)|Q_w(s)}{I_{zz}t(y)} \tag{7}$$

$Q_w(y)$ is calculated using the following equation:

$$Q_w(y) = \frac{b}{2}((\frac{h}{2})^2 - (\frac{h-2t_f}{2})^2) + \frac{t_w}{2}((\frac{h-2t_f}{2})^2) - y^2) \tag{8}$$

Using the above equations, the flexural normal stress and shear stress were calculated for each rosette as functions of load P, with x representing the x-coordinate and y representing the y-coordinate.

Rosette	x [in]	y [in]	σ_x [psi]	τ_{xy} [psi]
9-10-11	21.75	1	$\frac{-4P}{I_{zz}}$	0
12-13-14	21.75	2	$\frac{-8P}{I_{zz}}$	0
15-16-17	21.75	1	$\frac{-4P}{I_{zz}}$	0

Table 8: Expression of normal flexural stress and shear stress as functions of load P

Using a transformation matrix, the stress and strain components were calculated at the locations of the rosettes, as listed in Tables 9, 10, 11.

P_m [lb]	ϵ_x $(*10^{-6})$	ϵ_y $(*10^{-6})$	γ_{xy} $(*10^{-6})$	σx [psi]	σ_y [psi]	τ_{xy} [psi]
0	1	1	0	0	0	0
1000	-32.6	12.6	2.83	-370.5	0	0
2000	-67.2	26.2	4.95	-740.9	0	0
3002	-104.6	39.6	6.36	-1111	0	0
4002	-140.2	52.2	7.07	-1482	0	0
4998	-177.2	63.2	7.07	-1852	0	0
6000	-211.6	72.6	7.07	-2223	0	0
7000	-248.7	83.7	5.66	-2593	0	0
8000	-284.8	92.8	3.54	-2964	0	0

Table 9: Calculated stress and strain for strain gauges 9, 10, and 11.

P_m [lb]	ϵ_z $(*10^{-6})$	ϵ_x $(*10^{-6})$	γ_{xz} $(*10^{-6})$	σz [psi]	σ_x [psi]	τ_{xz} [psi]
0	1	1	0	0	0	0
1000	24.0	-69.0	-5.50	0	-740.9	0
2000	48.0	-141.0	-9.50	0	-1482	0
3001	72.0	-211.0	-11.5	0	-2224	0
4001	95.0	-279.0	13.0	0	-2964	0
5002	119.0	-346.0	-12.5	0	-3706	0
6001	143.0	-413.0	-11.0	0	-4446	0
7002	168.0	-480.0	-8.00	0	-5188	0
8000	192.0	-543.0	-3.50	0	-5927	0

Table 10: Calculated stress and strain for strain gauges 12, 13, and 14.

P_m [lb]	ϵ_x ($*10^{-6}$)	ϵ_y ($*10^{-6}$)	γ_{xy} ($*10^{-6}$)	σx [psi]	σ_y [psi]	τ_{xy} [psi]
0	0	0	-1	0	0	0
1000	-44.0	9.00	-7.50	-370.5	0	0
2000	-87.0	18.0	-12.5	-740.9	0	0
3001	-129.0	26.0	-17.5	-1111	0	0
4001	-168.0	35.0	-20.5	-1482	0	0
5002	-206.0	45.0	-22.5	-1852	0	0
6001	-241.0	54.0	-23.5	-2223	0	0
7002	-274.0	64.0	-22.0	-2593	0	0
8000	-306.0	73.0	-20.5	-2964	0	0

Table 11: Calculated stress and strain for strain gauges 15, 16, and 17.

Using another transformation matrix, the Young's modulus and Poisson's ratio of the material was calculated for each rosette, as shown in Table 12, alongside their 95% confidence intervals, with E being the Young's modulus, ν being Poisson's ratio, and CI_E and CI_ν being their respective confidence intervals. They are also compared to the published values for Aluminum 6061-T6511, which is 69 GPa for Young's modulus and 0.33 for Poisson's ratio [2]. PE_E is the percent error for Young's modulus, and PE_ν is the percent error for Poisson's ratio.

Rosette	E [GPa]	ν	CI_E [GPa]	CI_ν	PE_E [%]	PE_ν [%]
9-10-11	72.16	0.3422	[71.91, 72.92]	[0.3326, 0.3663]	4.6	3.7
12-13-14	74.37	0.3483	[73.56, 74.86]	[0.3434, 0.3513]	7.8	5.5
15-16-17	64.65	0.2037	[61.20, 66.15]	[0.0681, 0.2334]	6.3	38.3

Table 12: Comparison of rosettes' Young's modulus and Poisson's ratio with published values

The values for the 9-10-11 rosette and the 12-13-14 rosette are fairly similar, although the percent error of both the Young's modulus and Poisson's ratio are higher for the 12-13-14 rosette. The Young's modulus and Poisson's ratio of the 9-10-11 rosette and the 12-13-14 rosette are also both higher than the published values. In contrast, the Young's modulus and Poisson's ratio of the 15-16-17 are lower than the published values, and the percent error for its Poisson's ratio is a lot higher than the other rosettes. Its 95% confidence intervals also have a larger range than those of the other two rosettes.

CONCLUSIONS

Based on the data obtained, it is evident that values for strain gauges rosettes 9-10-11 9 and 15-16-17 11 are not the same, despite being placed at the same location (on opposite sides) of the beam. The strain gauge rosette 9-10-11 has a orientation of [-45,0,45] relative to the z axis, while the 15-16-17 rosette has an orientation of [-112.5, -67.5, -22.5] relative to the x axis. Thus, different strain gauge orientations affect how deformation is measured.

However, despite the difference in strain measurements, after matrix transformation and calculations using Hooke's law and stress-strain relationships, the calculated stress values were the same. This can be proved to be true using the stress equation 3. Since strain gauges rosettes 9-10-11 and 15-16-17 have the same distance from the neutral axis, are at the same x-location, and experience the same pure bending moment, the two rosettes undergo the same stress.

Furthermore, plane stress is a condition where out-of-plane stress, in this case σ_z, is zero. From the data gathered during the lab and calculations performed thereafter as table 10 illustrates, it is evident that σ_z is zero. The reason why the region where the strain gauge rosettes are placed experience plane stress is because the loads applied act in the xy-plane only 3, resulting in non-zero stress component σ_x, as illustrated in Tables 9-11.

Equally important, the effect of web thickness on bending stress and strain is worth mentioning. Based on the shear force and bending moment diagrams 3, it is clear that there is only pure bending at the location where the strain gauge rosettes are placed. However, shear strain is nonzero. The flexural stress equation 3 depends on the bending moment M, distance from neutral axis y, and moment of inertia of the beam I_{zz}. I_{zz} is defined by equation 4, where t_w is the thickness of weber. From this equation, it is fair to conclude that the web thickness does not significantly influence the bending stress. Since bending stress is barely affected by weber thickness, following Hooke's law, t_w will also have little effect on bending strain.

Similarly, it is also important to take into consideration what effects flange thickness t_f has on

bending stress and strain. From Equation 4, it is clear that flange thickness has a higher level of contribution to the moment of inertia of the beam. Therefore, as flange thickness is increased, bending stress is decreased according to the flexural stress equation 3. Thus, thicker flanges reduce the bending stress a material experiences. Since increasing flange thickness reduces stress, it also reduces strain and the level of deformation the beam undergoes. As a result, thicker flanges mean that the beam experiences less deformation and strain in general.

Additionally, it is important to note that the experimentally determined Young's modulus E and Poisson's ratio ν differ across strain gauge rosettes 12. This variation arises from differences in strain gauge orientations, which affect the measured strain values. Consequently, the calculated Young's modulus and Poisson's ratio also exhibit slight variations.

The values of Poisson's ratio ν and Young's modulus E for all three rosettes fell within their respective 95% confidence intervals. With one exception, the percent error for both ν and E remained within 8% of the expected values, as shown in Tables 12. The exception was ν for rosette 15-16-17, which exhibited a percent error of approximately 38%. Although this deviation is significant, it still lies within the 95% confidence interval. Potential sources of error include rounding in MATLAB calculations and minor coding inaccuracies, as the experiment was conducted in a virtual environment, with any discrepancies arising from data processing.

REFERENCE LIST

[1] NewcastleUniversity. "Moments (mechanics)." (2025), [Online]. Available: `https://www.ncl.ac.uk/webtemplate/ask-assets/external/maths-resources/mechanics/moments-main/moments.html` (visited on 03/06/2025).

[2] MakeItFrom. "6061-t6511 aluminum." (2020), [Online]. Available: `https://www.makeitfrom.com/material-properties/6061-T6511-Aluminum` (visited on 03/06/2025).

[3] OpenAI. "ChatGPT (March 7th, 2025 Version)," [Online]. Available: https://openai.com/chatgpt (visited on 03/07/2025).

DATA SHEETS

Load [lb]	Measured Load [lb]	Measured Strain (x10-6) Box 1, Ch.1 (Strain Gauge 9)	Measured Strain (x10-6) Box 1, Ch.2 (Strain Gauge 10)	Measured Strain (x10-6) Box 1, Ch.3 (Strain Gauge 11)
0	0	1	1	1
1000	1000	8	4	-28
2000	2000	16	9	-57
3000	3002	23	14	-88
4000	4002	29	19	-117
5000	4998	33	23	-147
6000	6000	36	26	-175
7000	7000	39	31	-204
8000	8000	40.00	35.00	-232.00

Lab Proceedure step 3: Readings from Virtual Lab Interface. This data set is also used for plotting in Data Reduction step 1

Figure 4: Measured Strain Values for Gauges 9-10-11

Load [lb]	Measured Load [lb]	Measured Strain (x10-6) Box 2, Ch.1 (Strain Gauge 12)	Measured Strain (x10-6) Box 2, Ch.2 (Strain Gauge 13)	Measured Strain (x10-6) Box 2, Ch.3 (Strain Gauge 14)
0	0	1.00	1.00	1.00
1000	1000	-17.00	24.00	-28.00
2000	2000	-37.00	48.00	-56.00
3000	3001	-58.00	72.00	-81.00
4000	4001	-79.00	95.00	-105.00
5000	5002	-101.00	119.00	-126.00
6000	6001	-124.00	143.00	-146.00
7000	7002	-148.00	168.00	-164.00
8000	8000	-172.00	192.00	-179.00

Lab Proceedure step 3: Readings from Virtual Lab Interface. This data set is also used for plotting in Data Reduction step 1

Figure 5: Measured Strain Values for Gauges 12-13-14

Load [lb]	Measured Load [lb]	Measured Strain (x10-6) Box 3, Ch.1 (Strain Gauge 15)	Measured Strain (x10-6) Box 3, Ch.2 (Strain Gauge 16)	Measured Strain (x10-6) Box 3, Ch.3 (Strain Gauge 17)
0	0.00	0.00	1.00	0.00
1000	1,000.00	9.00	-10.00	-44.00
2000	2,000.00	18.00	-22.00	-87.00
3000	3,001.00	26.00	-34.00	-129.00
4000	4,001.00	35.00	-46.00	-168.00
5000	5,002.00	45.00	-58.00	-206.00
6000	6,001.00	54.00	-70.00	-241.00
7000	7,002.00	64.00	-83.00	-274.00
8000	8,000.00	73.00	-96.00	-306.00
	Lab Proceedure step 3: Readings from Virtual Lab Interface. This data set is also used for plotting in Data Reduction step 1			

Figure 6: Measured Strain Values for Gauges 15-16-17

APPENDIX

The following MATLAB code was used to calculate the strain in the x direction ϵ_x , strain in the y direction ϵ_y, and the shear strain in the xy direction γ_x .

```

%% You need to run this code Twice
% Strain gauge rosette transformation matrix for strain gauges on the xy-plane
% (strain gauge rosette 9-10-11 and 15-16-17)

% In the space below, import an Excel or CSV file that contains the
% measured strain values from the virtual lab interface.
StrainData = readmatrix("Lab5_Tables_Template_fx.xlsx");

% Define the orientations of the strain gauges A, B and C with respect to the
    x-coordinate
thetaA = -112.5; % Angle A with respect to the x-axis [degrees]
thetaB = -67.5; % Angle B with respect to the x-axis [degrees]
thetaC = -22.5; % Angle C with respect to the x-axis [degrees]

% Referencing the imported Excel or CSV file, define a matrix that
```

```
% states the measured strain values for each loading step for a strain gauge rosette.
% Hint: This should be a 9 x 3 matrix.
strain_gauge_rosette_data = StrainData(1:9, 3:5); % Strain measurement matrix [microstrain (10^-6)]

% Transformation matrix calculations
strain_component_table= zeros(9,3); % Defining a matrix to store strain components for each loading step
for i= 1:9 % Looping through each loading step
    gauge_strain= transpose(strain_gauge_rosette_data(i,:)); % Defining measured strains for a loading step
    transform_matrix = [cosd(thetaA)^2, sind(thetaA)^2, 2*cosd(thetaA)*sind(thetaA);
    cosd(thetaB)^2, sind(thetaB)^2, 2*cosd(thetaB)*sind(thetaB);
    cosd(thetaC)^2, sind(thetaC)^2, 2*cosd(thetaC)*sind(thetaC)]; % Transofrmation matrix
% Strain components epsilon_x, epsilon_y and gamma_yx
    strain_component = transform_matrix\gauge_strain; % Calculating strain components for a loading step
    disp(['For load: ', num2str((i-1)*1000), ' lb, epsilon_x= ', num2str(strain_component(1)), ', epsilon_y= ', num2str(strain_component(2)), ' , gamma_xy= ', num2str(strain_component(3))]);
    strain_component_table(i,:)= strain_component; % Storing strain components in matrix
end

strain_component_table % Outputting matrix with stored strain components (units of microstrain) for each loading step



%% You should only need to run this once!
```

```
% Strain gauge rosette transformation matrix for strain gauges on the xz-plane
% (strain gauge rosette 12-13-14)

% In the space below, import an Excel or CSV file that contains the
% measured strain values from the virtual lab interface.
Gauges_12_13_14_Data= readmatrix('Lab5_Tables_Template.xlsx', 'Sheet', 'Strain
    Gauges 12, 13, 14');

% Define the orientations of the strain gauges A, B and C with respect to the
   x-coordinate
thetaA = -45; % Angle A with respect to the z-axis [degrees]
thetaB = 0; % Angle B with respect to the z-axis [degrees]
thetaC = 45; % Angle C with respect to the z-axis [degrees]

% Referencing the imported Excel or CSV file, define a matrix that
% states the measured strain values for each loading step for a strain gauge
   rosette.
% Hint: This should be a 9 x 3 matrix.
strain_gauge_rosette_data = Gauges_12_13_14_Data(1:9, 3:5); % Strain
   measurement matrix [microstrain (10^-6)]

% Transformation matrix calculations
strain_component_table= zeros(9,3); % Defining a matrix to store strain
   components for each loading step
for i= 1:9 % Looping through each loading step
    gauge_strain= transpose(strain_gauge_rosette_data(i,:)); % Defining
   measured strains for a loading step
    transform_matrix = [cosd(thetaA)^2, sind(thetaA)^2, 2*cosd(thetaA)*sind(
   thetaA);
    cosd(thetaB)^2, sind(thetaB)^2, 2*cosd(thetaB)*sind(thetaB);
    cosd(thetaC)^2, sind(thetaC)^2, 2*cosd(thetaC)*sind(thetaC)]; %
   Transofrmation matrix
% Strain components epsilon_x, epsilon_y and gamma_yx
```

```
    strain_component = transform_matrix\gauge_strain; % Calculating strain components for a loading step
    disp(['For load: ', num2str((i-1)*1000), ' lb, epsilon_z= ', num2str(strain_component(1)), ' , epsilon_x= ', num2str(strain_component(2)), ' , gamma_xz= ', num2str(strain_component(3))]);
    strain_component_table(i,:)= strain_component; % Storing strain components in matrix
end

strain_component_table % Outputting matrix with stored strain components (units of microstrain) for each loading step

%% You need to run this code Twice
% Strain gauge rosette transformation matrix for strain gauges on the xy-plane
% (strain gauge rosette 9-10-11 and 15-16-17)

% In the space below, import an Excel or CSV file that contains the
% measured strain values from the virtual lab interface.
StrainData = readmatrix("Lab5_Tables_Template_151617.xlsx", 'Sheet', 'Strain Gauges 15, 16, 17');

% Define the orientations of the strain gauges A, B and C with respect to the x-coordinate
thetaA = -90; % Angle A with respect to the x-axis [degrees]
thetaB = -45; % Angle B with respect to the x-axis [degrees]
thetaC = 0; % Angle C with respect to the x-axis [degrees]

% Referencing the imported Excel or CSV file, define a matrix that
% states the measured strain values for each loading step for a strain gauge rosette.
% Hint: This should be a 9 x 3 matrix.
strain_gauge_rosette_data = StrainData(1:9, 3:5); % Strain measurement matrix [microstrain (10^-6)]
```

```

% Transformation matrix calculations
strain_component_table= zeros(9,3); % Defining a matrix to store strain components for each loading step
for i= 1:9 % Looping through each loading step
    gauge_strain= transpose(strain_gauge_rosette_data(i,:)); % Defining measured strains for a loading step
    transform_matrix = [cosd(thetaA)^2, sind(thetaA)^2, 2*cosd(thetaA)*sind(thetaA);
    cosd(thetaB)^2, sind(thetaB)^2, 2*cosd(thetaB)*sind(thetaB);
    cosd(thetaC)^2, sind(thetaC)^2, 2*cosd(thetaC)*sind(thetaC)]; % Transofrmation matrix
% Strain components epsilon_x, epsilon_y and gamma_yx
    strain_component = transform_matrix\gauge_strain; % Calculating strain components for a loading step
    disp(['For load: ', num2str((i-1)*1000), ' lb, epsilon_x= ', num2str(strain_component(1)), ', epsilon_y= ', num2str(strain_component(2)), ' , gamma_xy= ', num2str(strain_component(3))]);
    strain_component_table(i,:)= strain_component; % Storing strain components in matrix
end

strain_component_table % Outputting matrix with stored strain components (units of microstrain) for each loading step
```

The following code was use to estimate the Young's Modulus and respective 95% confidence interval. Moreover, the code was used to estimate ν and its 95% confidence interval.

```
%% You need to run the function twice
% Estimation of Young's modulus and Poisson's ratio for strain gauges
% located on the xy-surface.


```

```
function beam_model_xy()
% In the space below, import an Excel or CSV file that contains the
% strain and stress components for the strain gauge.
Gauge_Data= readmatrix("Lab5_Tables_Template_mihk.xlsx", 'Sheet', 'Strain
    Gauges 15, 16, 17');

% Referencing the imported Excel or CSV file, define vectors for each
% strain and stress component.
sigma_x = Gauge_Data(1:9,9); % Stress in x-direction [psi]
tau_xy = Gauge_Data(1:9,11); % Shear stress in xy-plane [psi]
epsilon_x = Gauge_Data(1:9,6); % Strain in x-direction [microstrain]
epsilon_y = Gauge_Data(1:9,7); % Strain in y-direction [microstrain]
gamma_xy = Gauge_Data(1:9,8); % Shear strain in xy-plane [microstrain]
n = length(sigma_x); % number of data points
numBootstraps = 1000; % number of bootstrap samples
% Initial guess for E and nu
E0 = 1;
nu0 = 0.3;
% Define objective function
objfun = @(x)...
sum((sigma_x-x(1)*(epsilon_x + x(2)*epsilon_y)./(1-x(2)^2)).^2 ...
+ (-x(1)*(x(2)*epsilon_x + epsilon_y)./(1-x(2)^2)).^2 ...
+ (tau_xy-x(1)*gamma_xy/2./(1+x(2))).^2);
objfun_bootstrap = @(x,sigma_x,tau_xy,epsilon_x,epsilon_y,gamma_xy)...
sum((sigma_x-x(1)*(epsilon_x + x(2)*epsilon_y)./(1-x(2)^2)).^2 ...
+ (-x(1)*(x(2)*epsilon_x + epsilon_y)./(1-x(2)^2)).^2 ...
+ (tau_xy-x(1)*gamma_xy/2./(1+x(2))).^2);
% Define constraints for nu (-1<nu<0.5 for most materials)
options = optimoptions(@fmincon,'Algorithm','interior-point');
lb = [0,-1]; % lower bounds
ub = [Inf,0.5]; % upper bounds
% Initialize array to hold bootstrap estimates
bootstrap_estimates = zeros(numBootstraps,2);
```

```
% Call fmincon to minimize objective function
x = fmincon(objfun,[E0;nu0],[],[],[],[],lb,ub,[],options);
% Perform bootstrap estimation
for i = 1:numBootstraps
    % Generate bootstrap sample
    ind = randi(n,n,1);
    sigma_x_b = sigma_x(ind);
    tau_xy_b = tau_xy(ind);
    epsilon_x_b = epsilon_x(ind);
    epsilon_y_b = epsilon_y(ind);
    gamma_xy_b = gamma_xy(ind);
    % Call fmincon to minimize objective function for bootstrap sample
    x_b = fmincon(@(x)...
    objfun_bootstrap(x,sigma_x_b,tau_xy_b,epsilon_x_b,epsilon_y_b,gamma_xy_b),...
    [E0;nu0],[],[],[],[],lb,ub,[],options);
    bootstrap_estimates(i,:) = x_b;
end
% Compute confidence intervals
alpha = 0.05; % 95% confidence interval
E_conf_int = quantile(bootstrap_estimates(:,1),[alpha/2, 1-alpha/2]);
nu_conf_int = quantile(bootstrap_estimates(:,2),[alpha/2, 1-alpha/2]);
% Display the optimal values of E and nu
disp(['Optimal value of E: ', num2str(x(1)), ' Mpsi'])
disp(['Optimal value of nu: ', num2str(x(2))])
% Display the confidence intervals
disp(['95% confidence interval for E: [', num2str(E_conf_int(1)), ', ', num2str(E_conf_int(2)), ']', ' Mpsi'])
disp(['95% confidence interval for nu: [', num2str(nu_conf_int(1)), ', ', num2str(nu_conf_int(2)), ']'])
end
```

```
%% You need to run the function once
```

```
% Estimation of Young's modulus and Poisson's ratio for strain gauges
% located on the xz-surface.

function beam_model_xz()
% In the space below, import an Excel or CSV file that contains the
% strain and stress components for the strain gauge.
Gauge_Data= readmatrix("Lab5_Tables_Template_updog.xlsx", 'Sheet','Strain
    Gauges 12, 13, 14');


% Referencing the imported Excel or CSV file, define vectors for each
% strain and stress component.
sigma_x = Gauge_Data(1:9,10); % Stress in the x-direction [psi]
tau_xz = Gauge_Data(1:9,11); % Shear stress in the xz-plane [psi]
epsilon_z = Gauge_Data(1:9,6); % Strain in the z-direction [microstrain]
epsilon_x = Gauge_Data(1:9,7); % Strain in the x-direction [microstrain]
gamma_xz = Gauge_Data(1:9,8); % Shear strain in the xz-plane [microstrain]
n = length(sigma_x); % numer of data points
numBootstraps = 1000; % number of bootstrap samples
% Initial guess for E and nu
E0 = 1;
nu0 = 0.3;
% Define objective function
objfun = @(x)...
sum((sigma_x-x(1)*(epsilon_x + x(2)*epsilon_z)./(1-x(2)^2)).^2 ...
+ (-x(1)*(x(2)*epsilon_x + epsilon_z)./(1-x(2)^2)).^2 ...
+ (tau_xz-x(1)*gamma_xz/2./(1+x(2))).^2);
objfun_bootstrap = @(x,sigma_x,tau_xz,epsilon_z,epsilon_x,gamma_xz)...
sum((sigma_x-x(1)*(epsilon_x + x(2)*epsilon_z)./(1-x(2)^2)).^2 ...
+ (-x(1)*(x(2)*epsilon_x + epsilon_z)./(1-x(2)^2)).^2 ...
+ (tau_xz-x(1)*gamma_xz/2./(1+x(2))).^2);
% Define constraints for nu (-1<nu<0.5 for most materials)
options = optimoptions(@fmincon,'Algorithm','interior-point');
```

```
lb = [0,-1]; % lower bounds
ub = [Inf,0.5]; % upper bounds
% Initialize array to hold bootstrap estimates
bootstrap_estimates = zeros(numBootstraps,2);
% Call fmincon to minimize objective function
x = fmincon(objfun,[E0;nu0],[],[],[],[],lb,ub,[],options);
% Perform bootstrap estimation
for i = 1:numBootstraps
    % Generate bootstrap sample
    ind = randi(n,n,1);
    sigma_x_b = sigma_x(ind);
    tau_xz_b = tau_xz(ind);
    epsilon_z_b = epsilon_z(ind);
    epsilon_x_b = epsilon_x(ind);
    gamma_xz_b = gamma_xz(ind);
    % Call fmincon to minimize objective function for bootstrap sample
    x_b = fmincon(@(x)...
    objfun_bootstrap(x,sigma_x_b,tau_xz_b,epsilon_z_b,epsilon_x_b,gamma_xz_b)
    ,...
    [E0;nu0],[],[],[],[],lb,ub,[],options);
    bootstrap_estimates(i,:) = x_b;
end
% Compute confidence intervals
alpha = 0.05; % 95% confidence interval
E_conf_int = quantile(bootstrap_estimates(:,1),[alpha/2, 1-alpha/2]);
nu_conf_int = quantile(bootstrap_estimates(:,2),[alpha/2, 1-alpha/2]);
% Display the optimal values of E and nu
disp(['Optimal value of E: ', num2str(x(1)), ' Mpsi'])
disp(['Optimal value of nu: ', num2str(x(2))])
% Display the confidence intervals
disp(['95% confidence interval for E: [', num2str(E_conf_int(1)), ', ',
    num2str(E_conf_int(2)), ']', ' Mpsi'])
disp(['95% confidence interval for nu: [', num2str(nu_conf_int(1)), ', ',
```

```
    num2str(nu_conf_int(2)), ']']])
end
```